

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



TRABAJO GRUPAL - PATRONES DE ARQUITECTURA
IMPLEMENTACIÓN DEL PATRÓN MAPREDUCE EN SISTEMAS DISTRIBUIDOS

ASIGNATURA

SISTEMAS DISTRIBUIDOS

GRUPO C

DOCENTE

Mg. MARIBEL MOLINA BARRIGA

INTEGRANTES

BEDREGAL PEREZ DANIEL

JARA MAMANI MARIEL ALISSON

MESTAS ZEGARRA CHRISTIAN RAUL

QUISPE CONDORI ALVARO RAUL

SEQUEIROS CONDORI LUIS GUSTAVO

AREQUIPA - PERÚ

2026

Tabla de Contenido

1. Definición del Patrón MapReduce	3
1.1. Ventajas del Patrón MapReduce	3
1.2. Desventajas del Patrón MapReduce	3
2. Diseño de la Arquitectura	3
3. Código Fuente de la Implementación	5
3.1. Definición del Servicio gRPC	5
3.2. Fase de Mapeo (Worker)	5
3.3. Fase de Reducción y Orquestación (Master)	5
3.4. Capturas de Ejecución	6
4. Conclusiones sobre la Implementación	7
4.1. Comparativa con Otras Arquitecturas	8
5. Bibliografía	8

Implementación del Patrón MapReduce para Conteo de Palabras Distribuido

1. Definición del Patrón MapReduce

MapReduce es un modelo de programación y una implementación asociada para procesar y generar grandes conjuntos de datos de forma paralela y distribuida en un clúster (Dean & Ghemawat, 2004). El usuario especifica una función Map que procesa un par clave/valor para generar un conjunto de pares clave/valor intermedios, y una función Reduce que combina todos los valores intermedios asociados con la misma clave intermedia (Dean & Ghemawat, 2008).

1.1. Ventajas del Patrón MapReduce

- **Escalabilidad:**

Permite procesar petabytes de datos utilizando hardware convencional al distribuir la carga (Dean & Ghemawat, 2004).

- **Tolerancia a fallos:**

El sistema gestiona automáticamente los fallos de los nodos, reasignando tareas si un trabajador no responde.

- **Abstracción:**

El desarrollador solo debe preocuparse por la lógica de negocio (Map y Reduce), dejando la gestión de red y paralelismo al framework (Dean & Ghemawat, 2008).

1.2. Desventajas del Patrón MapReduce

- **No es apto para tiempo real:**

La sobrecarga de orquestación y el manejo de archivos intermedios introducen latencia (Condie et al., 2010).

- **Iteraciones costosas:**

Algoritmos que requieren múltiples pasadas sobre los mismos datos (como el entrenamiento de modelos de ML) son ineficientes debido a la escritura constante en disco o red entre fases (Ekanayake et al., 2010).

2. Diseño de la Arquitectura

Para abordar el problema del conteo de frecuencia de palabras, se ha diseñado una arquitectura basada en el modelo Maestro-Trabajador (Master-Worker), donde los componentes se comunican a través de gRPC.

1. Master (Maestro):

- Expone un endpoint HTTP para recibir el texto a procesar.
- Divide el texto en fragmentos manejables (chunks).

- Mantiene un registro de los trabajadores (Workers) disponibles.
- Orquesta la distribución de tareas y recolecta los resultados parciales para la reducción final.

2. Worker (Trabajador):

- Se registra con el Maestro al iniciar su ejecución.
- Implementa el servicio gRPC para realizar el conteo local de un fragmento de texto.
- Retorna un diccionario de frecuencias al Maestro.

3. Infraestructura:

- Se utiliza Docker Compose para desplegar un nodo maestro y múltiples nodos trabajadores en una red virtual.



Figura 1. Arquitectura del sistema de conteo distribuido

3. Código Fuente de la Implementación

En esta sección se detallan las partes fundamentales del código, utilizando el componente de bloques de código importado de la librería del curso.

3.1. Definición del Servicio gRPC

El contrato de comunicación entre el maestro y los trabajadores se define mediante Protocol Buffers.

```
syntax = "proto3";

package wordcount.v1;

message CountWordsRequest {
  string text = 1;
}

message CountWordsResponse {
  map<string, int32> counts = 1;
}

service WordCountWorkerService {
  rpc CountWords(CountWordsRequest) returns (CountWordsResponse);
}
```

3.2. Fase de Mapeo (Worker)

La lógica del trabajador se centra en la limpieza del texto y el conteo de palabras únicas en su fragmento asignado.

```
import re
from collections import Counter
from typing import Dict

def run_map(text_chunk: str) -> Dict[str, int]:
    """
    Get a text chunk, clean it, and return a dictionary with word counts.
    """
    clear_text = text_chunk.lower()
    words = re.findall(r"\b[a-zA-ZÉÍÓÚ]+\b", clear_text)
    return dict(Counter(words))
```

3.3. Fase de Reducción y Orquestación (Master)

El maestro distribuye los fragmentos de forma concurrente y luego consolida los resultados en un solo diccionario.

```
def distribute_map_tasks(chunks: list[str]) -> list[dict]:
    """
    Distributes map tasks across registered workers using gRPC.
    """
    def process_chunk(args):
        i, chunk = args
        worker = WORKERS[i % len(WORKERS)]
        try:
            with grpc.insecure_channel(worker, options=options) as channel:
```

```
stub = pb2_grpc.WordCountWorkerServiceStub(channel)
request = pb2.CountWordsRequest(text=chunk)
response = stub.CountWords(request, timeout=60)
return dict(response.counts)
except Exception as e:
    return {}

with concurrent.futures.ThreadPoolExecutor(
    max_workers=min(20, len(WORKERS) * 2)
) as executor:
    results = list(executor.map(process_chunk, enumerate(chunks)))
return [r for r in results if r]
```

La función de reducción une los conteos parciales:

```
from typing import List

def run_reduce(mapped_data: List[dict]) -> dict:
    """
    Take a list of dictionaries (partial word counts from the map phase) and combine them into a single dictionary with
    total counts.
    """
    total_counts = {}
    for partial_counts in mapped_data:
        for word, count in partial_counts.items():
            if word in total_counts:
                total_counts[word] += count
            else:
                total_counts[word] = count
    return total_counts
```

3.4. Capturas de Ejecución

A continuación se muestran las capturas que validan el correcto funcionamiento del sistema en sus diferentes interfaces.

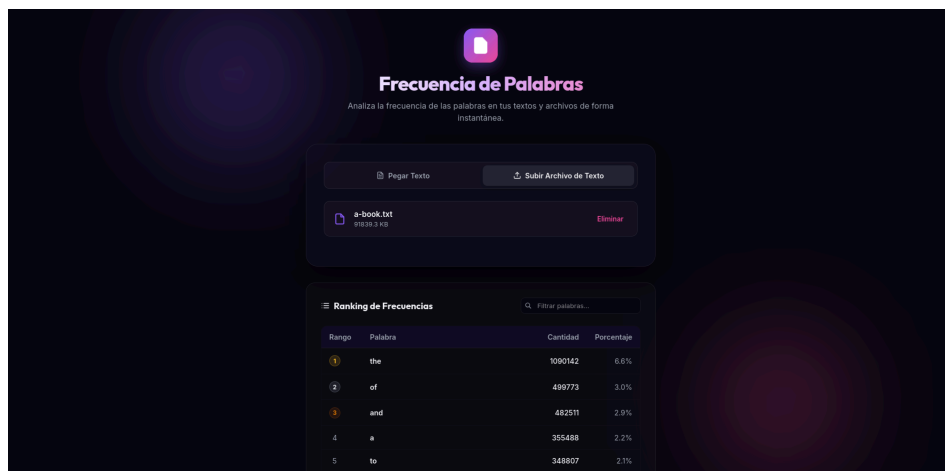


Figura 2. Interfaz Web para el análisis de texto

```
> uv run cli.py -f a-short-book.txt

=====
RANKING DE FRECUENCIA DE PALABRAS
=====
Rango | Palabra | Frecuencia
-----|-----|-----
1 | the | 4
2 | lazy | 3
3 | jumps | 3
4 | fast | 3
5 | quick | 2
6 | brown | 2
7 | fox | 2
8 | yes | 2
9 | dog | 2
10 | it | 1
11 | over | 1
12 | is | 1
13 | very | 1
=====

Total de palabras procesadas: 27
Palabras únicas encontradas: 13
Tiempo de análisis: 6.416552998416591 ms
=====

>
```

Figura 3. Ejecución y resultados vía Cliente CLI

```
> docker compose up --scale worker=8
[*] up 1/1
✓ Container mapreduce-master-1 Running 0.0s
WARN[0000] could not start menu, an error occurred while starting: Error while reading terminfo data:termbox: unsupporte
d terminal
Attaching to master-1, worker-1, worker-2, worker-3, worker-4, worker-5, worker-6, worker-7, worker-8
worker-3 | [Worker] Starting gRPC server...
master-1 | 172.21.0.3 - - [24/May/2026 22:37:12] "POST /register HTTP/1.1" 200 -
worker-3 | [Worker] Successfully registered at http://master:9000/register
worker-3 | [Worker] gRPC server started on 1101cb2d5987:50051
worker-5 | [Worker] Starting gRPC server...
master-1 | 172.21.0.4 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-5 | [Worker] Successfully registered at http://master:9000/register
worker-5 | [Worker] gRPC server started on f8faeb5ed805:50051
worker-7 | [Worker] Starting gRPC server...
master-1 | 172.21.0.5 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-7 | [Worker] Successfully registered at http://master:9000/register
worker-7 | [Worker] gRPC server started on 040b1dc9358:50051
worker-2 | [Worker] Starting gRPC server...
master-1 | 172.21.0.6 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-2 | [Worker] Successfully registered at http://master:9000/register
worker-2 | [Worker] gRPC server started on 7dc1acb208a0:50051
worker-8 | [Worker] Starting gRPC server...
master-1 | 172.21.0.7 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-8 | [Worker] Successfully registered at http://master:9000/register
worker-8 | [Worker] gRPC server started on a90846ff5f02:50051
worker-4 | [Worker] Starting gRPC server...
master-1 | 172.21.0.8 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-4 | [Worker] Successfully registered at http://master:9000/register
worker-4 | [Worker] gRPC server started on 4446d7e0d8c7:50051
worker-6 | [Worker] Starting gRPC server...
master-1 | 172.21.0.9 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-6 | [Worker] Successfully registered at http://master:9000/register
worker-6 | [Worker] gRPC server started on 70d66c8a16f0:50051
worker-1 | [Worker] Starting gRPC server...
master-1 | 172.21.0.10 - - [24/May/2026 22:37:13] "POST /register HTTP/1.1" 200 -
worker-1 | [Worker] Successfully registered at http://master:9000/register
worker-1 | [Worker] gRPC server started on fd7b21f439e1:50051
master-1 | [Master] Processing request: 137 characters
master-1 | [Master] Distributing 27 words across 8 workers (chunk size: 20)
master-1 | [Master] Sending chunk 0 to 1101cb2d5987:50051
master-1 | [Master] Sending chunk 1 to f8faeb5ed805:50051
worker-5 | [Worker] Received text chunk of size 32
worker-3 | [Worker] Received text chunk of size 103
master-1 | 172.21.0.11 - - [24/May/2026 22:37:18] "POST /api/count HTTP/1.1" 200 -
"Gracefully Stopping... press Ctrl+C again to force
Container mapreduce-master-1 Stopping
Container mapreduce-worker-7 Stopping
Container mapreduce-worker-5 Stopping
Container mapreduce-worker-2 Stopping
Container mapreduce-worker-4 Stopping
Container mapreduce-worker-8 Stopping
Container mapreduce-worker-6 Stopping
Container mapreduce-worker-3 Stopping
```

Figura 4. Logs del sistema mostrando la distribución entre Master y Worker

4. Conclusiones sobre la Implementación

La implementación del patrón MapReduce resuelve eficazmente el problema del conteo de palabras al permitir que el procesamiento se realice de forma distribuida. Esto evita que un solo nodo se sature al intentar cargar archivos de texto extremadamente grandes en memoria, permitiendo que el sistema escale linealmente con el número de trabajadores.

Sin embargo, el sistema podría fallar si el nodo Maestro deja de funcionar, ya que es el único orquestador de las tareas. Asimismo, una red lenta podría degradar el rendimiento si el tiempo de transferencia de los fragmentos de texto supera al tiempo de procesamiento local.

4.1. Comparativa con Otras Arquitecturas

- **Pipeline:**

Sería eficiente para transformaciones de datos en flujo, pero no maneja nativamente la agregación masiva necesaria para el conteo global de palabras (Condie et al., 2010).

- **Pub/Sub:**

Facilitaría la escalabilidad de trabajadores, pero la sincronización de la fase final (Reduce) sería compleja sin un coordinador central (Kleppmann, 2017).

- **Broker:**

Centralizaría la comunicación, pero el overhead de mensajes podría ser mayor al flujo directo gRPC entre maestro y trabajadores (Kleppmann, 2017).

- **P2P (Peer-to-Peer):**

Ofrecería una alta disponibilidad sin puntos de fallo únicos, pero la lógica para recolectar y reducir los datos de todos los pares requeriría un protocolo de consolidación mucho más complejo (Marozzo et al., 2011).

5. Bibliografía

- Condie, T., Conway, N., Alvaro, P., Hellerstein, J. M., Elmeleegy, K., & Sears, R. (2010,). MapReduce Online. *7th USENIX Symposium on Networked Systems Design and Implementation (OSDI 10)*. <https://www.usenix.org/conference/nsdi-10/mapreduce-online>
- Dean, J., & Ghemawat, S. (2004,). MapReduce: Simplified Data Processing on Large Clusters. *6th Symposium on Operating Systems Design & Implementation (OSDI 04)*. <https://www.usenix.org/conference/osdi-04/mapreduce-simplified-data-processing-large-clusters>
- Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107–113. <https://doi.org/10.1145/1327452.1327492>
- Ekanayake, J., Li, H., Zhang, B., Gunarathne, T., Bae, S.-H., Qiu, J., & Fox, G. (2010,). Twister: A Runtime for Iterative MapReduce. *1st International Workshop on Mapreduce and Its Applications (MAPREDUCE '10)*. <https://doi.org/10.1145/1851476.1851486>
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- Marozzo, F., Talia, D., & Trunfio, P. (2011). P2P-MapReduce: Parallel data processing in dynamic cloud environments. *Journal of Computer and System Sciences*, 78(5), 1382–1402. <https://doi.org/10.1016/j.jcss.2011.12.016>